

ASSIGNMENT 10.4

AI-ASSISTED CODING

Code Review and Quality: Using AI to Improve Code Quality and Readability

Submitted By

Name: Bitla Sairaj

Roll Number: 2503A52162

Branch: CSE Ai & ML

Section: 06

Date: 08-09-26

1. INTRODUCTION

This assignment demonstrates the use of AI tools as a code reviewer to improve code quality, performance, readability, security, and maintainability. It consists of five tasks: syntax and code-quality review, performance-oriented optimization, readability and maintainability refactoring, secure coding and reliability review, and an AI-based automated code review report.

Each task follows the same workflow: an intentionally flawed Python program is written, an AI tool (GitHub Copilot / Cursor AI) is prompted to review it, the AI's findings and its corrected or optimized code are recorded, and the corrected program is executed to verify that the fix works as expected. The goal is to show AI being used as a code reviewer rather than only as a code generator.

2. TOOLS USED

- Python
- Cursor AI / GitHub Copilot
- VSCode / Cursor
- Terminal

3. TASK 1

3.1 Objective

Use AI to identify and correct syntax errors, indentation problems, incorrect variable names, and faulty function calls in a Python script, following PEP 8 practices.

3.2 Original Code

```
def calculate_average(numbers)
    total=0

    for num in numbers:
        total+=num

    average = total / len(number)
    return averge

numbers = [10,20,30,40,50]

result = calculate_average(numbers)
print("Average:",result)
```

3.3 AI Prompt

Review the following Python code as a senior Python developer.

Identify all:

1. Syntax errors
2. Indentation errors
3. Incorrect variable names
4. Incorrect function calls
5. Structural/code-quality issues

Do not immediately rewrite the code.

First provide a numbered list of every problem, explain why it is a problem, and then provide a corrected version.

After the corrected code, explain every change individually and mention any PEP 8 improvements.

3.4 AI Review

The AI identified that the function definition was missing a colon, that the code referenced an undefined variable (number instead of numbers), that the return variable was misspelled (average instead of average), and that the function body's indentation needed to be corrected for the code to run at all.

3.5 Identified Issues

Issue	Problem	Correction
Function definition	Missing :	Added:
Variable name	number doesn't exist	Changed to numbers
Variable name	average misspelled	Changed to average
Indentation	Function body incorrectly structured	Corrected indentation
Execution	Function needed to execute after correction	Verified using Python

3.6 Corrected Code

```
def calculate_average(numbers):
    total = 0

    for num in numbers:
        total += num
```

```
average = total / len(numbers)
return average
```

```
numbers = [10, 20, 30, 40, 50]
```

```
result = calculate_average(numbers)
print("Average:", result)
```

3.7 Explanation of Fixes

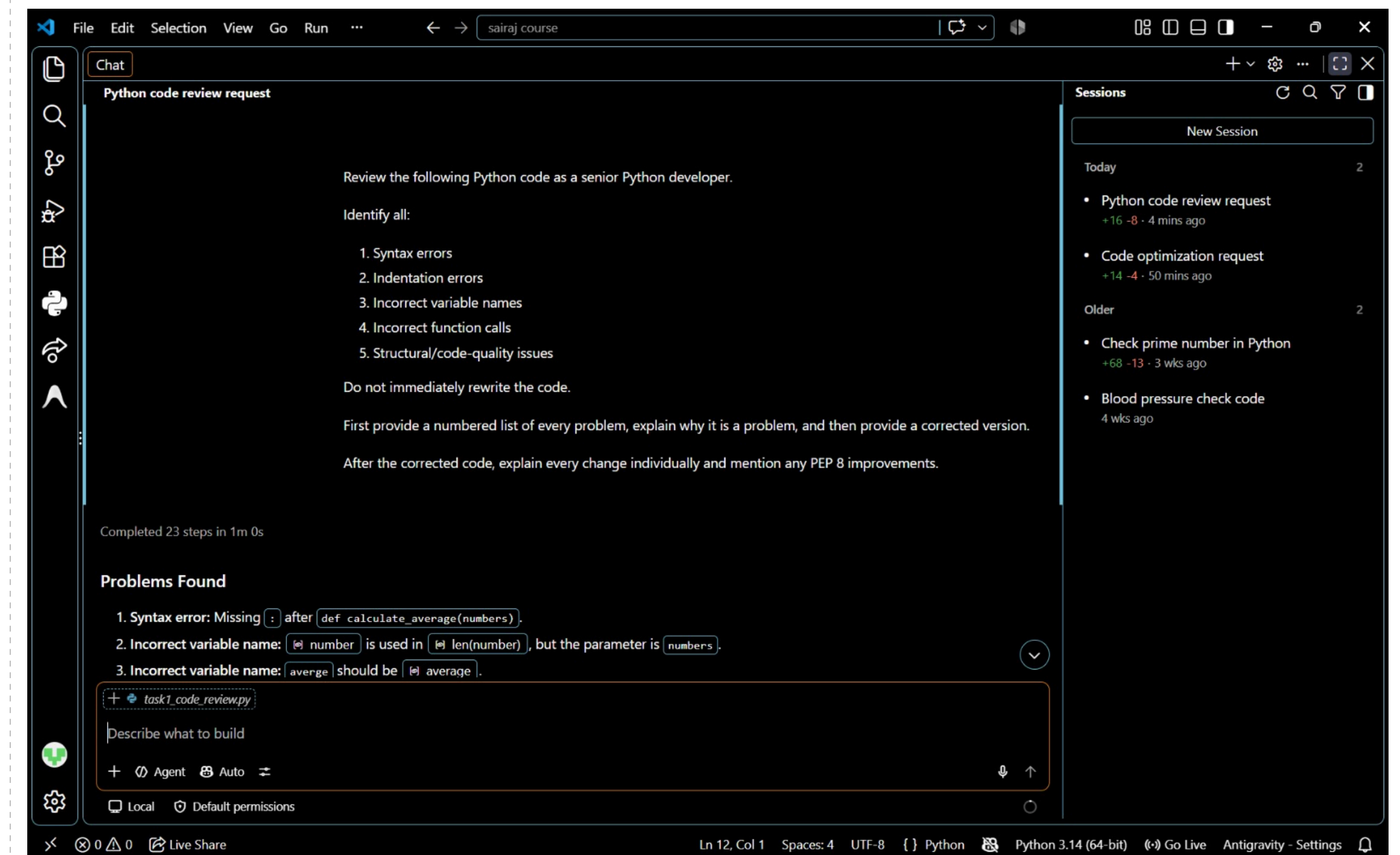
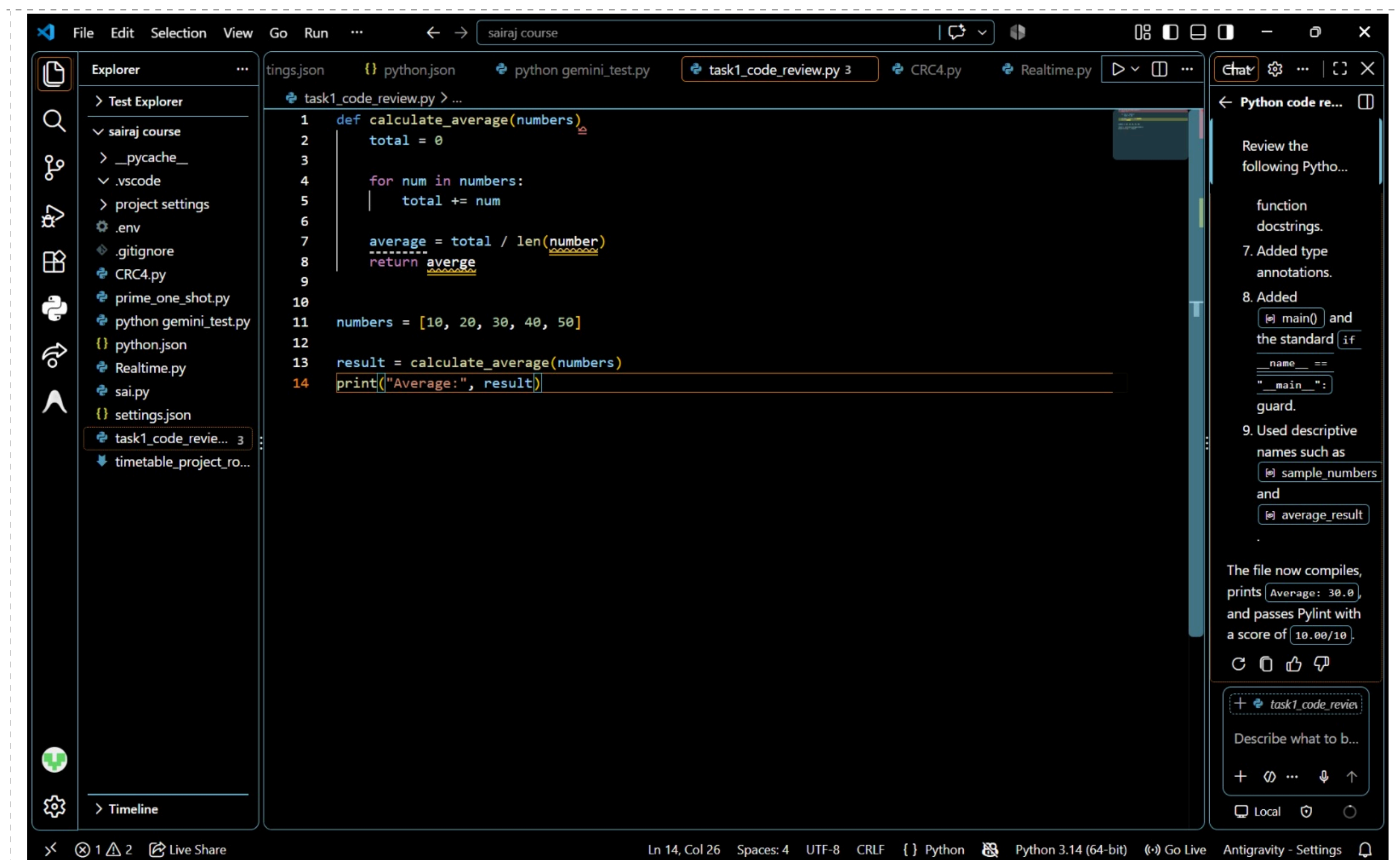
- Added the missing colon (:) at the end of the function definition so Python recognizes the function body.
- Corrected the undefined variable `number` to `numbers` so `len()` operates on the correct list.
- Fixed the misspelled variable `average` to `average` so the return statement references the correct name.
- Corrected the indentation of the function body to follow PEP 8's 4-space indentation standard.

3.8 Execution Result

Running `python task1_code_review.py` produces the following output:

```
Average: 30.0
```

3.9 Screenshots



FileEditSelectionViewGoRun...<→sairaj course

Chat

Python code review request

Review the following Python code as a senior Python developer....

Completed 23 steps in 1m 0s

Problems Found

1. Syntax error: Missing `:` after `def calculate_average(numbers)`.

2. Incorrect variable name: `number` is used in `len(number)`, but the parameter is `numbers`.

3. Incorrect variable name: `average` should be `average`.

4. Indentation: No indentation error was present; the function body used valid four-space indentation.

5. Function calls: The call to `calculate_average(numbers)` was correctly structured. It was only unusable because the function definition had a syntax error.

6. Empty input issue: An empty list would cause `ZeroDivisionError`.

7. Code quality: The original code lacked type hints and documentation.

8. Naming/style: Module-level execution and variable naming triggered Pylint warnings.

Corrected Version

The corrected code is in `task1_code_review.py`:

"""Calculate the average of a collection of numbers."""

def calculate_average(values: list[float]) -> float:

+ task1_code_review.py

Describe what to build

+ Agent Auto

Local Default permissions

Sessions

New Session

Today2

Python code review request

+16 -8 · 4 mins ago

Code optimization request

+14 -4 · 50 mins ago

Older2

Check prime number in Python

+68 -13 · 3 wks ago

Blood pressure check code

4 wks ago

Ln 12, Col 1Spaces: 4UTF-8PythonPython 3.14 (64-bit)Go LiveAntigravity - Settings

FileEditSelectionViewGoRun...<→sairaj course

Explorer

Test Explorer

sairaj course

__pycache__

.vscode

project settings

.env

.gitignore

CRC4.py

prime_one_shot.py

python_gemini_test.py

python.json

Realtime.py

sai.py

settings.json

task1_code_review.py

timetable_project_ro...

Timeline

task1_code_review.py > ...

1 """Calculate the average of a collection of numbers."""

2

3

4 def calculate_average(values: list[float]) -> float:

5 """Return the arithmetic mean of the supplied numbers."""

6 if not values:

7 | raise ValueError("numbers must not be empty")

8

9 total = 0

10 for number in values:

11 | total += number

12

13 average = total / len(values)

14 return average

15

16

17 def main() -> None:

18 """Calculate and display the average for sample input."""

19 sample_numbers = [10, 20, 30, 40, 50]

20 average_result = calculate_average(sample_numbers)

21 print("Average:", average_result)

22

23

24 if __name__ == "__main__":

25 | main()

26

Python code re...

Review the following Python...

but the parameter is `numbers`.

3. Incorrect variable name: `average` should be `average`.

4. Indentation: No indentation error was present; the function body used valid four-space indentation.

5. Function calls: The call to `calculate_average` was correctly structured. It was only unusable because the function definition had...

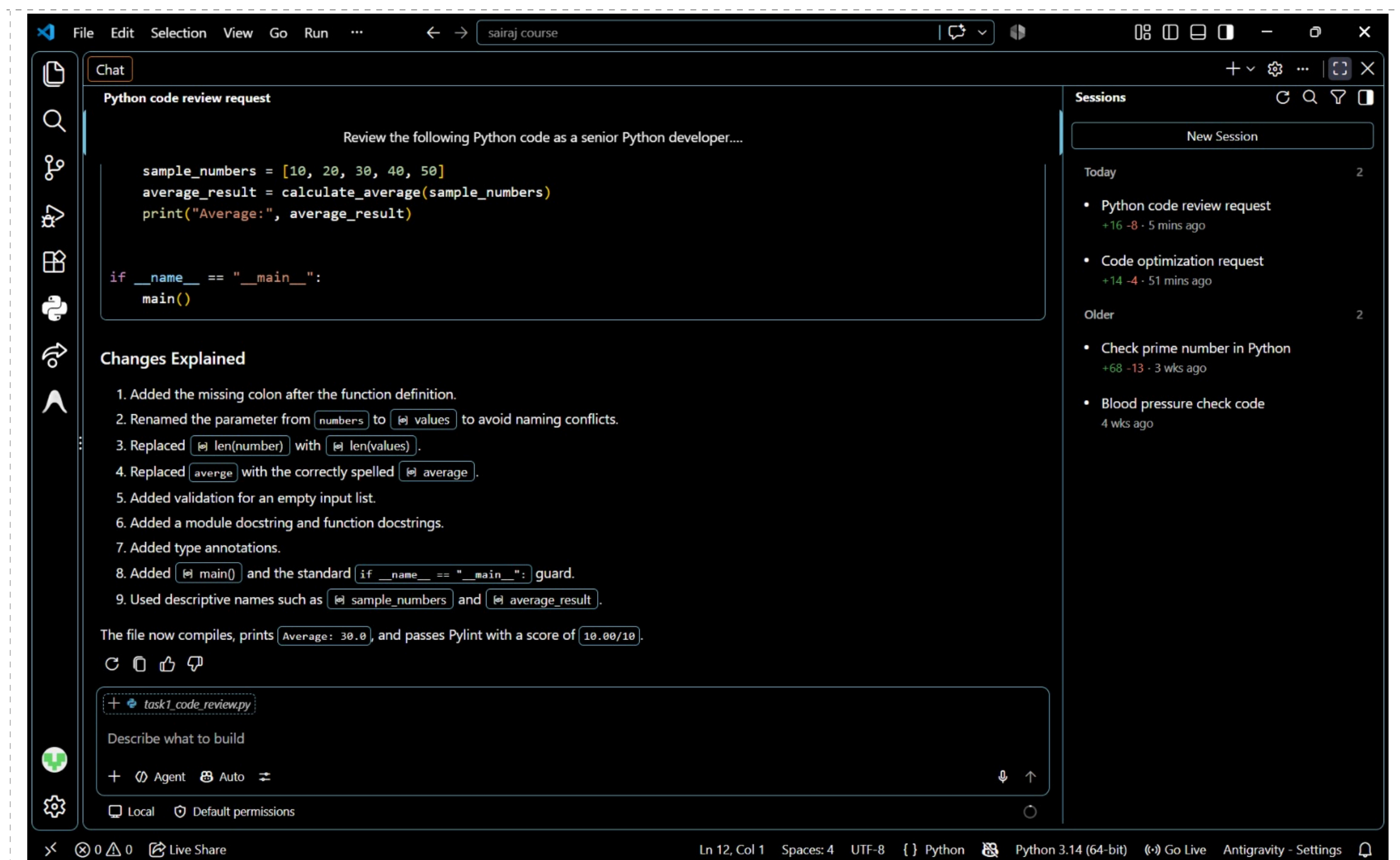
+ task1_code_review.py

Describe what to b...

+ ...

Local

Ln 26, Col 1Spaces: 4UTF-8PythonPython 3.14 (64-bit)Go LiveAntigravity - Settings



4. TASK2

4.1 Objective

Use AI to identify the performance bottleneck in a duplicate-detection function and optimize it using an appropriate hash-based data structure.

4.2 Original Code

```
def find_duplicates(numbers):
    duplicates = []

    for i in range(len(numbers)):
        for j in range(i + 1, len(numbers)):
            if numbers[i] == numbers[j]:
                if numbers[i] not in duplicates:
                    duplicates.append(numbers[i])

    return duplicates

numbers = [1, 2, 3, 4, 2, 5, 6, 3, 7, 8, 2]

print("Duplicates:", find_duplicates(numbers))
```

4.3 AI Prompt

Act as a senior Python performance engineer.

Review the following duplicate-detection function.

1. Identify the performance bottleneck.
2. Explain the time complexity of the original implementation.
3. Refactor it using an appropriate hash-based data structure such as a set.
4. Preserve the same expected output.
5. Explain the time and space complexity of the optimized implementation.
6. Follow PEP8.

Do not change the expected behavior.

4.4 Performance Analysis

The AI identified that the nested-loop structure compares every element of the list with every other element, resulting in a quadratic number of comparisons. As the input size grows, the number of comparisons grows proportionally to n^2 , making the original implementation inefficient for large inputs.

The AI recommended replacing the nested loops with a hash-based approach using Python sets, which allow near constant-time membership checks (average case), reducing the overall time complexity.

4.5 Optimized Code

```
def find_duplicates(numbers):
    seen = set()
    duplicates = set()

    for number in numbers:
        if number in seen:
            duplicates.add(number)
        else:
            seen.add(number)

    return list(duplicates)
```

Note: because a standard Python set does not preserve insertion order, the order of elements in the returned duplicates list may differ from the original nested-loop version. If exact ordering must match, ask the AI to preserve insertion order (e.g. using a list + a lookup set) instead of a plain set.

4.6 Complexity Comparison

Version	Approach	Time Complexity	Space Complexity
Original	Nested loops (compare every pair)	$O(n^2)$	$O(n)$
Optimized	Hash set (seen / duplicates)	$O(n)$ average	$O(n)$

4.7 Execution Result

Running python task2_performance.py produces the same duplicate values as the original version (order may vary due to the use of a set):

```
Duplicates: [2, 3]
```

4.8 Screenshots



File Edit Selection View Go Run ...

sairaj course

Chat

Python duplicate detection optimization

Act as a senior Python performance engineer.

Review the following duplicate-detection function.

1. Identify the performance bottleneck.
2. Explain the time complexity of the original implementation.
3. Refactor it using an appropriate hash-based data structure such as a set.
4. Preserve the same expected output.
5. Explain the time and space complexity of the optimized implementation.
6. Follow PEP 8.

Do not change the expected behavior.

Completed 9 steps in 35s

Updated `task2_performance.py`.

- Replaced nested comparisons with `seen` and `duplicates_seen` sets.
- Preserved duplicate order and output: `Duplicates: [2, 3]`.
- Original complexity: $O(n^2)$ time and $O(n)$ space.
- Optimized complexity: $O(n)$ average time and $O(n)$ space.
- Verified with `python -m py_compile task2_performance.py`.

+ task2_performance.py

Describe what to build

+ Agent Auto

Local Default permissions

Sessions

New Session

Today3

- Python duplicate detection optimization
+8 -6 · 3 mins ago
- Python code review request
+16 -8 · 12 mins ago
- Code optimization request
+14 -4 · 58 mins ago

Older2

- Check prime number in Python
+68 -13 · 3 wks ago
- Blood pressure check code
4 wks ago

Ln 18, Col 1 Spaces: 4 UTF-8 CRLF {} Python Python 3.14 (64-bit) Go Live Antigra - Settings

File Edit Selection View Go Run ...

sairaj course

python.json python gemini_test.py task2_performance.py CRC4.py

task2_performance.py > ...

```
1 def find_duplicates(numbers):
2     duplicates = []
3
4     for i in range(len(numbers)):
5         for j in range(i + 1, len(numbers)):
6             if numbers[i] == numbers[j]:
7                 if numbers[i] not in duplicates:
8                     duplicates.append(numbers[i])
9
10    return duplicates
11
12
13 numbers = [1, 2, 3, 4, 2, 5, 6, 3, 7, 8, 2]
14
15 print("Duplicates:", find_duplicates(numbers))
```

Problems Output Debug Console Terminal Ports

PS D:\sairaj course> & "C:\Program Files\Python314\python.exe" "d:\sairaj cour |
• se/task2_performance.py"
Duplicates: [2, 3]
PS D:\sairaj course>

Python: tas...

1 Hidden ...

Ln 15, Col 47 Spaces: 4 UTF-8 CRLF {} Python Python 3.14 (64-bit) Go Live Antigra - Settings

Chat

Python duplicate detection optimization

Act as a senior Python performance engineer....

5. Explain the time and space complexity of the optimized implementation.
6. Follow PEP 8.

Do not change the expected behavior.

Completed 9 steps in 35s

Updated `task2_performance.py`.

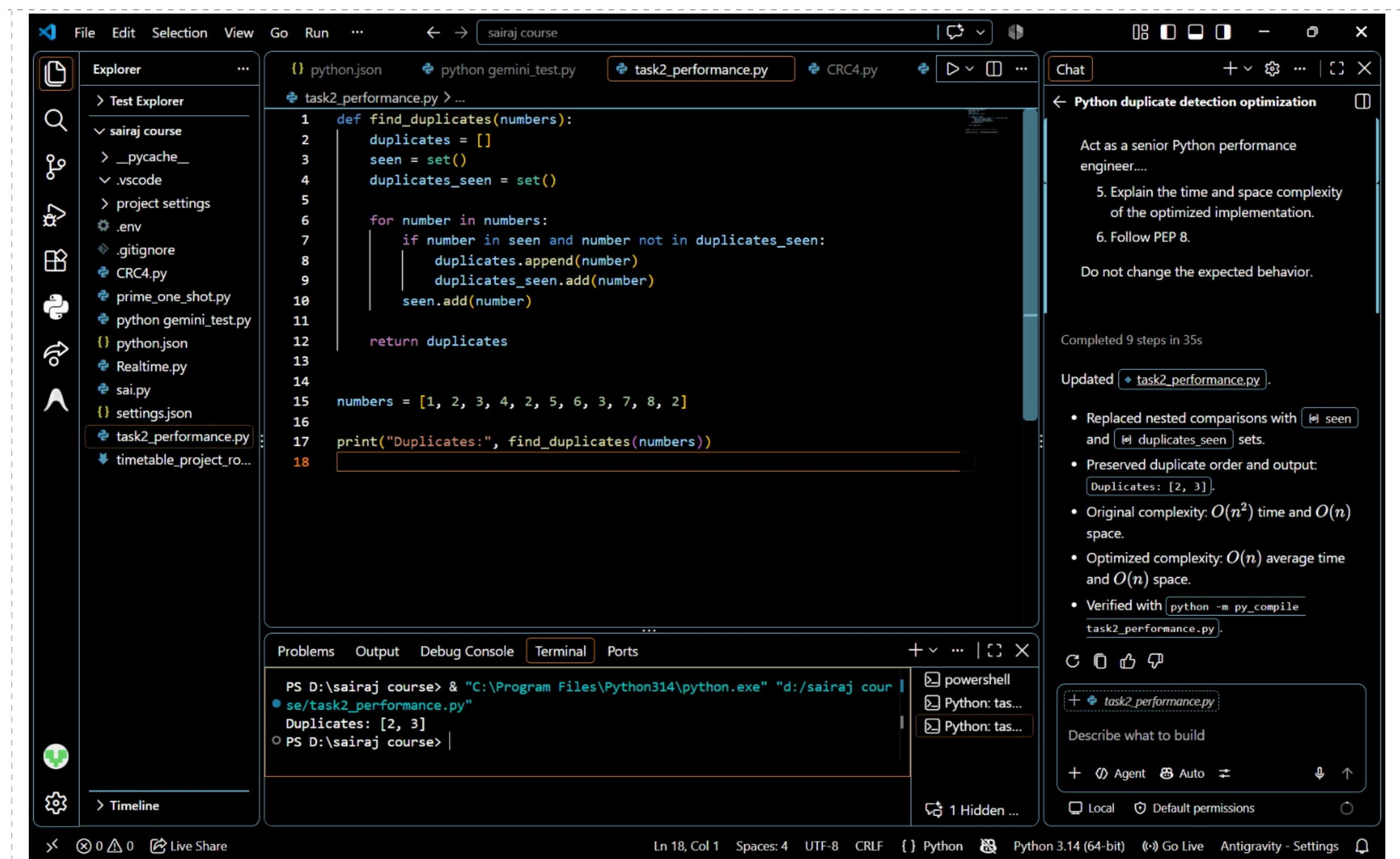
- Replaced nested comparisons with `seen` and `duplicates_seen` sets.
- Preserved duplicate order and output: `Duplicates: [2, 3]`.
- Original complexity: $O(n^2)$ time and $O(n)$ space.
- Optimized complexity: $O(n)$ average time and $O(n)$ space.
- Verified with `python -m py_compile task2_performance.py`.

+ task2_performance.py

Describe what to build

+ Agent Auto

Local Default permissions



5.TASK3

5.1Objective

Use AI to refactor a poorly written function for readability and maintainability by improving naming, formatting, indentation, and documentation while preserving its behavior.

5.2 Original Code

```
def f(a,b):  
    x=0  
    for i in a:  
        if i>b:  
            x=x+i  
    return x  
  
data=[10,20,30,40,50]  
limit=25  
print(f(data,limit))
```

5.3 AI Prompt

Act as a senior Python code reviewer.

Refactor the following Python function for readability and long-term maintainability.

Requirements:

- 1. Use descriptive function names.
- 2. Use descriptive variable names.
- 3. Apply PEP 8 formatting.
- 4. Correct indentation.
- 5. Add an appropriate docstring.
- 6. Add useful comments only where necessary.
- 7. Preserve the existing functionality and output.
- 8. Explain every major readability improvement.

First analyze the problems in the original code, then provide the refactored version.

5.4 Refactoring Analysis

The AI identified that the function and variable names (f, a, b, x, i) were cryptic and did not describe their purpose, that the code lacked a docstring, that indentation was inconsistent (single-space instead of 4-space), and that PEP 8 spacing conventions (e.g. spaces around operators and after commas) were not followed.

5.5 Refactored Code

```
def sum_values_above_limit(values, limit):
    """Return the sum of values greater than the specified limit."""
    total = 0

    for value in values:
        if value > limit:
            total += value

    return total

data = [10, 20, 30, 40, 50]
limit = 25

result = sum_values_above_limit(data, limit)
print(result)
```

5.6 PEP 8 Improvements

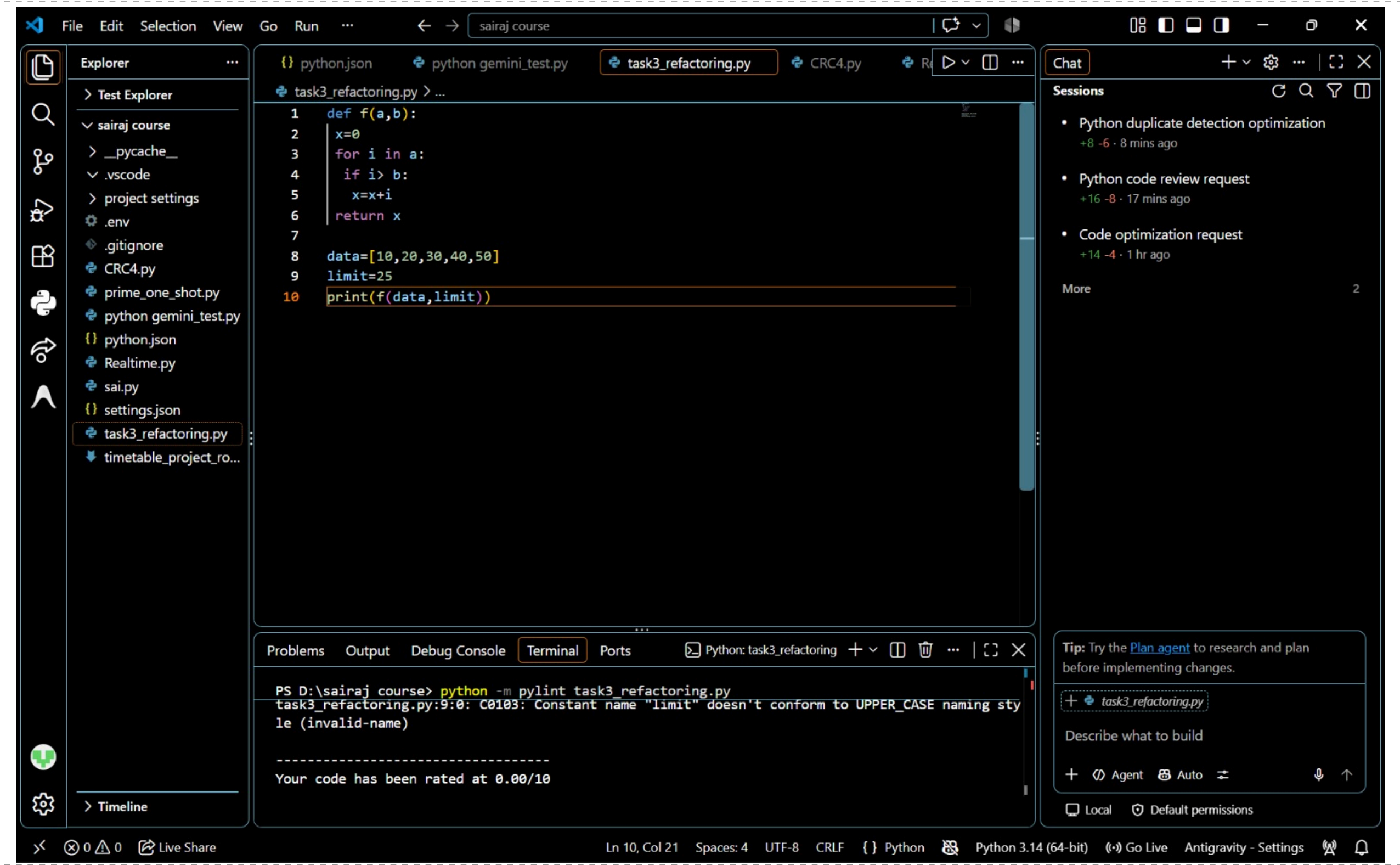
Before	After
--------	-------

Before	After
f()	sum_values_above_limit()
a	values
b	limit
x	total
Poor indentation	PEP 8 (4-space) indentation
No documentation	Added docstring
Poor spacing	Standard formatting (spaces around operators, blank lines)

5.7 Readability Improvements

Descriptive names (sum_values_above_limit, values, limit, total) make the function's purpose clear without needing to read its body. The docstring documents what the function returns. Consistent 4-space indentation and blank lines separating the function from the calling code make the program easier to scan and maintain.

5.8 Screenshots



FileEditSelectionViewGoRun...<→sairaj course

Chat

Python code refactoring request

Start Over

Act as a senior Python code reviewer.

Refactor the following Python function for readability and long-term maintainability.

Requirements:

1. Use descriptive function names.

2. Use descriptive variable names.

3. Apply PEP 8 formatting.

4. Correct indentation.

5. Add an appropriate docstring.

6. Add useful comments only where necessary.

7. Preserve the existing functionality and output.

8. Explain every major readability improvement.

First analyze the problems in the original code, then provide the refactored version.

Sorry, your request failed. Please try again.

Client Request Id: f977d9b9-5799-49dd-aa36-fad32b0348eb

GH Request Id: F9C0:111EB4:5ADD590:65F7E58:6A9FF3AD

Reason: Request Failed: 400 ("error": "The requested model is not")

Todos (4/4)

+ task3_refactoring.py

Describe what to build

+ Agent Auto

Local Default permissions

Sessions

New Session

Today 4

Python code refactoring request

+12 -10 · 13 mins ago

Python duplicate detection optimization

+8 -6 · 19 mins ago

Python code review request

+16 -8 · 28 mins ago

Code optimization request

+14 -4 · 1 hr ago

Older 2

Check prime number in Python

+68 -13 · 3 wks ago

Blood pressure check code

4 wks ago

< 0 0 Live ShareLn 16, Col 1Spaces: 4UTF-8[] PythonPython 3.14 (64-bit)Go LiveAntigravity - Settings

FileEditSelectionViewGoRun...<→sairaj course

Chat

Python code refactoring request

Act as a senior Python code reviewer....

Selected "Try Again"

Completed 12 steps in 8m 34s

Refactored task3_refactoring.py.

Major improvements:

Renamed f to sum_values_above_limit.

Replaced unclear names with values, threshold, value, and total.

Applied PEP 8 spacing and four-space indentation.

Added module and function docstrings.

Preserved the strict > comparison and output.

Used += for clearer accumulation.

Renamed the global threshold to THRESHOLD for Pylint compliance.

Validation passed:

Script output: 120

Pylint score: 10.00/10

5:17 PM • GPT-5.6 Luna • 1.0 credits

Todos (4/4)

+ task3_refactoring.py

Describe what to build

+ Agent Auto

Local Default permissions

Sessions

New Session

Today 4

Python code refactoring request

+12 -10 · 13 mins ago

Python duplicate detection optimization

+8 -6 · 19 mins ago

Python code review request

+16 -8 · 28 mins ago

Code optimization request

+14 -4 · 1 hr ago

Older 2

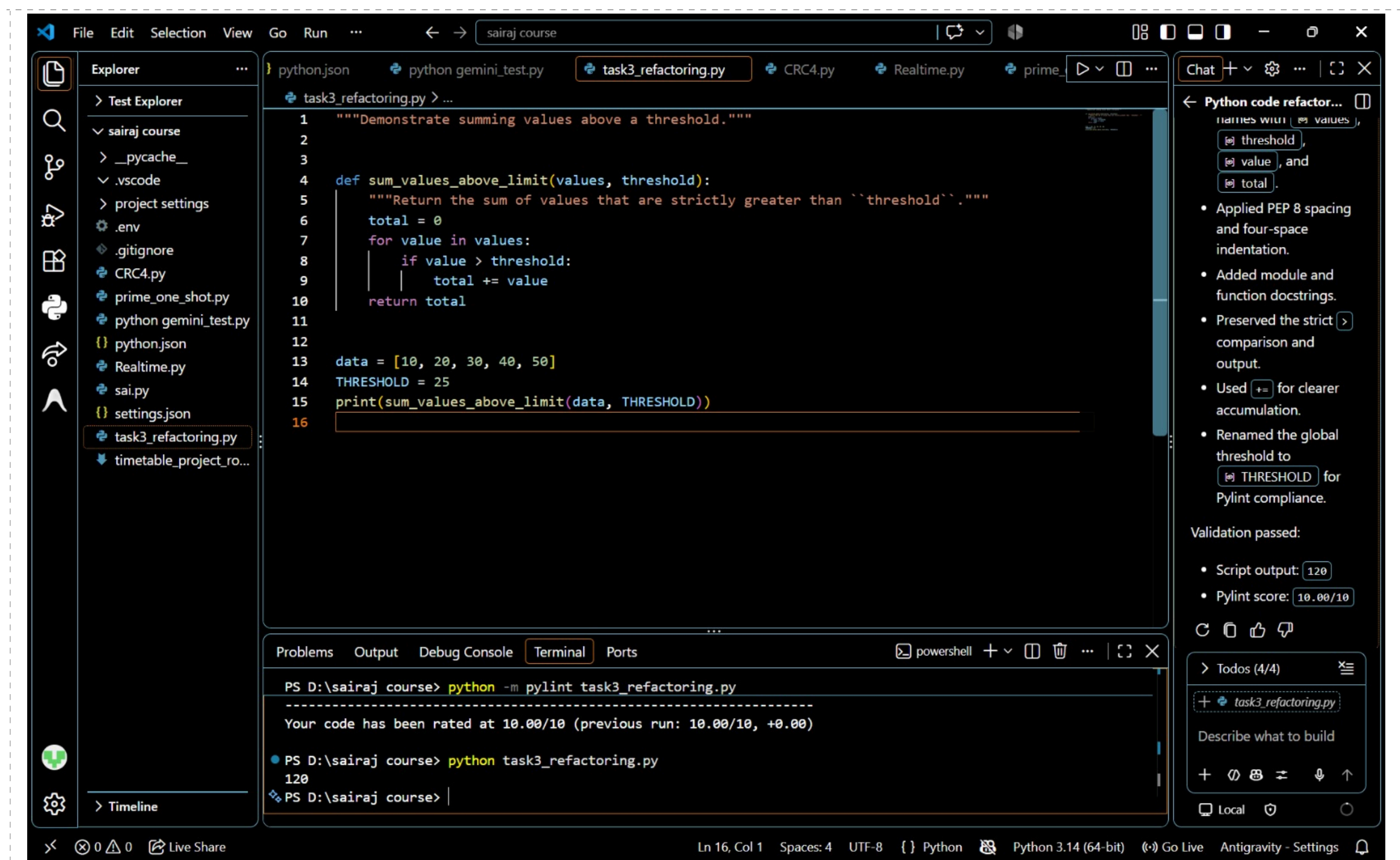
Check prime number in Python

+68 -13 · 3 wks ago

Blood pressure check code

4 wks ago

< 0 0 Live ShareLn 16, Col 1Spaces: 4UTF-8[] PythonPython 3.14 (64-bit)Go LiveAntigravity - Settings



Expected output after running the refactored code:

120

6. TASK 4

6.1 Objective

Use AI to perform a security review of a database-access function, identify SQL injection and other reliability problems, and refactor the code using secure coding practices.

6.2 Vulnerable Code

```
import sqlite3

def get_user(username):
    connection = sqlite3.connect("users.db")
    cursor = connection.cursor()

    query = "SELECT * FROM Users WHERE username = " + username + ""
```

```
cursor.execute(query)
```

```
result = cursor.fetchone()
```

```
connection.close()
```

```
return result
```

```
username = input("Enter username: ")
```

```
print(get_user(username))
```

6.3 AI Security Review Prompt

Act as a senior application security engineer reviewing Python backend code.

Analyze the following database function for security, reliability, and robustness problems.

Identify:

1. SQL injection vulnerabilities
2. Missing input validation
3. Resource-management problems
4. Missing exception handling
5. Other reliability concerns

Then refactor the code using secure coding practices.

Requirements:

- Use parameterized SQL queries.
- Validate the username input.
- Add appropriate try-except handling.
- Ensure database resources are safely handled.
- Do not expose sensitive implementation details to the user.
- Explain every security improvement.

6.4 Security Vulnerabilities

- SQL injection: the username is concatenated directly into the SQL string, allowing malicious input to alter the query.
- No input validation: the username is used exactly as entered, with no checks on length, emptiness, or content.
- Unsafe resource handling: the database connection is only closed on the normal path; if an exception occurs before `connection.close()`, the connection is never released.

- No exception handling: any database error (e.g. a locked or missing database file) crashes the program and may expose internal error details to the user.

6.5 Secure Code

```
import sqlite3

def get_user(username):
    if not username or len(username) > 50:
        raise ValueError("Invalid username")

    try:
        connection = sqlite3.connect("users.db")
        cursor = connection.cursor()

        cursor.execute(
            "SELECT * FROM users WHERE username=?",
            (username,)
        )

        result = cursor.fetchone()
        return result

    except sqlite3.Error:
        print("An error occurred while accessing the database.")
        return None

    finally:
        connection.close()

username = input("Enter username: ")
print(get_user(username))
```

6.6 Input Validation

The username is checked for emptiness and a maximum length (50 characters) before it is used. Invalid input raises a `ValueError` immediately, preventing malformed or excessively long input from reaching the database layer.

6.7 Exception Handling

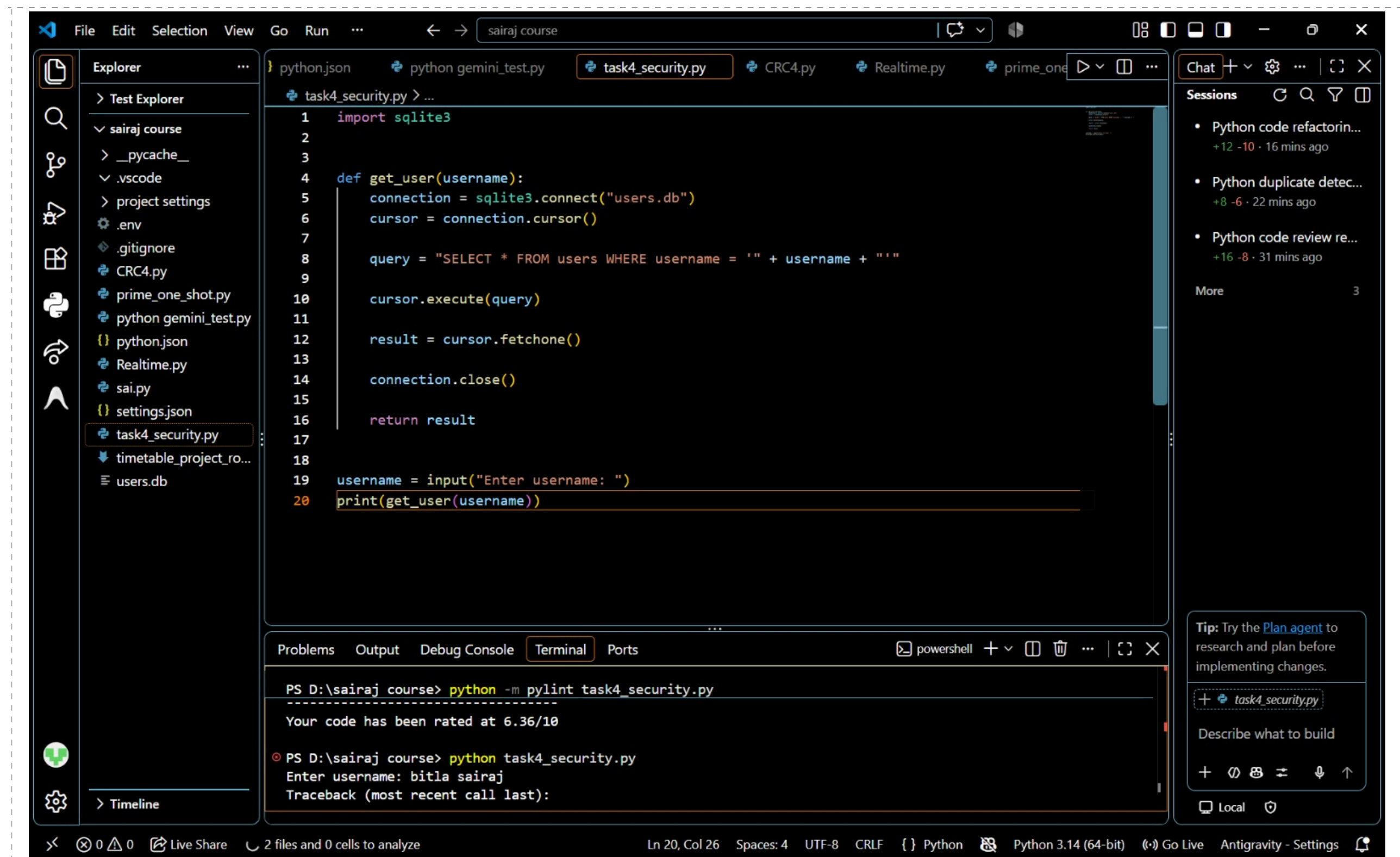
The database operations are wrapped in a try-except-finally block. The except clause catches `sqlite3.Error` and prints a generic message instead of exposing internal error details, while the finally block guarantees the connection is closed whether or not an error occurred.

6.8 Security Improvements

Before: user input flowed directly from `input()` into string concatenation and then into the database (User input → SQL string construction → database).

After: user input is first validated, then passed as a parameter to a parameterized query, which the database driver safely binds without allowing SQL syntax injection (User input → validation → parameterized query → database). This is the single most important fix, since parameterized queries eliminate SQL injection risk regardless of what the user types.

6.9 Screenshots



FileEditSelectionViewGoRun...<→sairaj course

Chat

Python backend security review

Start Over

Act as a senior application security engineer reviewing Python backend code.

Analyze the following database function for security, reliability, and robustness problems.

Identify:

1. SQL injection vulnerabilities

2. Missing input validation

3. Resource-management problems

4. Missing exception handling

5. Other reliability concerns

Then refactor the code using secure coding practices.

Requirements:

• Use parameterized SQL queries.

• Validate the username input.

• Add appropriate try-except handling.

• Ensure database resources are safely handled.

• Do not expose sensitive implementation details to the user.

• Explain every security improvement.

11:49

+ task4_security.py

Describe what to build

+ Agent Auto

Local Default permissions

Sessions

New Session

Yesterday5

Python backend security review

+30 -14 · 1 day ago

Python code refactoring request

+12 -10 · 1 day ago

Python duplicate detection optimization

+8 -6 · 1 day ago

Python code review request

+16 -8 · 1 day ago

Code optimization request

+14 -4 · 1 day ago

Older2

Check prime number in Python

+68 -13 · 3 wks ago

Blood pressure check code

4 wks ago

Ln 37, Col 1Spaces: 4UTF-8[] PythonPython 3.14 (64-bit)Go LiveAntigravity - Settings

FileEditSelectionViewGoRun...<→sairaj course

Chat

Python backend security review

Act as a senior application security engineer reviewing Python backend code....

• validate the username input.

• Add appropriate try-except handling.

• Ensure database resources are safely handled.

• Do not expose sensitive implementation details to the user.

• Explain every security improvement.

11:49 AM

Completed 12 steps in 39s

Refactored task4_security.py.

Security and reliability improvements:

• Replaced string-concatenated SQL with a parameterized query, preventing SQL injection.

• Validates username type, trims whitespace, enforces length limits, and allows only safe characters.

• Ensures both cursor and database connection close through `finally`, including failure paths.

• Adds SQLite exception handling and converts database errors into a generic application error.

• Prevents raw database rows and implementation details from being shown to users.

• Handles invalid input and database failures with a safe user-facing message.

• Avoids exposing sensitive record contents by reporting only "User found" or "User not found."

Validation completed successfully with `python -m py_compile task4_security.py`.

11:50 AM • GPT-5.6 Luna • 0.9 credits

+ task4_security.py

Describe what to build

+ Agent Auto

Local Default permissions

Sessions

New Session

Yesterday5

Python backend security review

+30 -14 · 1 day ago

Python code refactoring request

+12 -10 · 1 day ago

Python duplicate detection optimization

+8 -6 · 1 day ago

Python code review request

+16 -8 · 1 day ago

Code optimization request

+14 -4 · 1 day ago

Older2

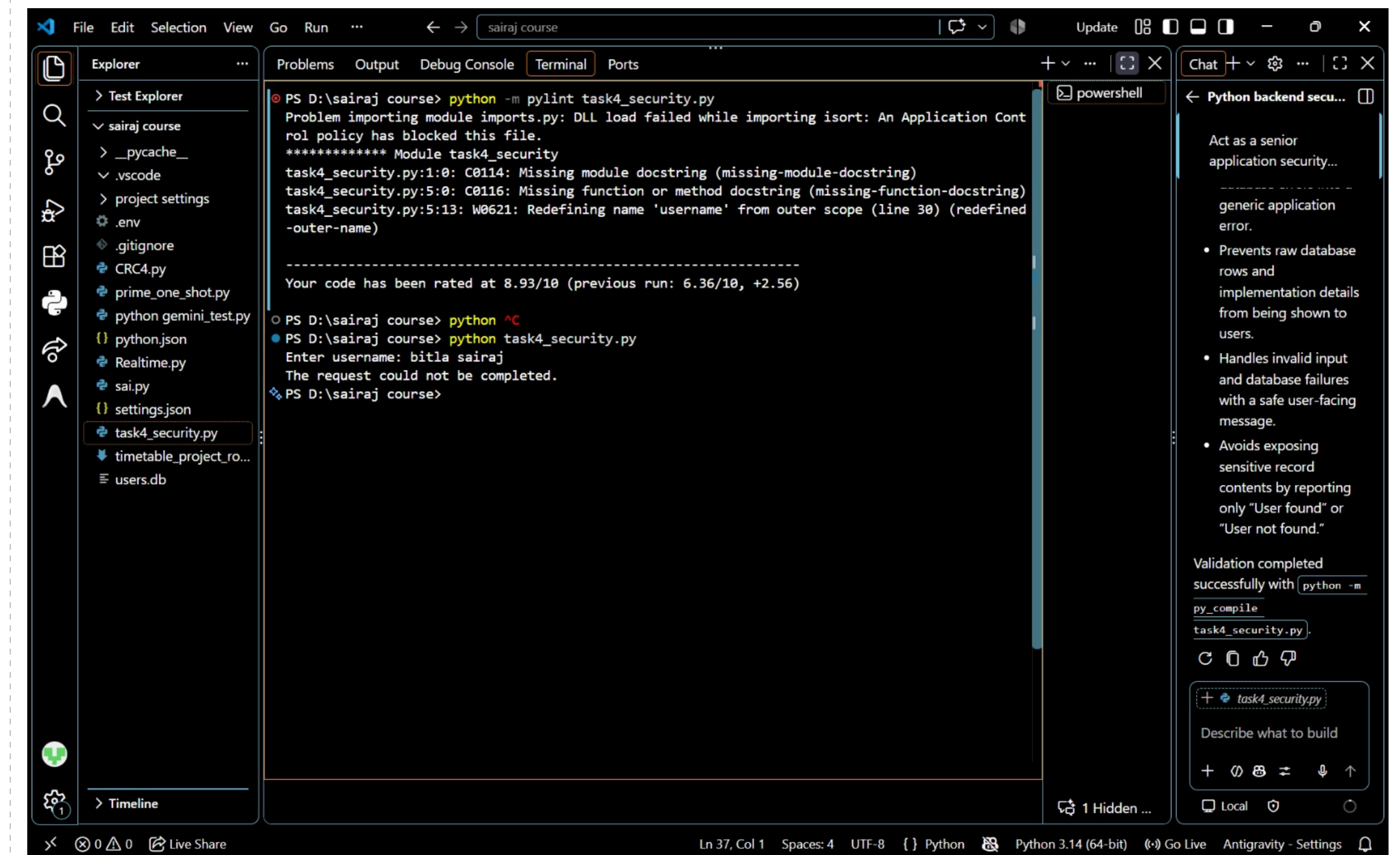
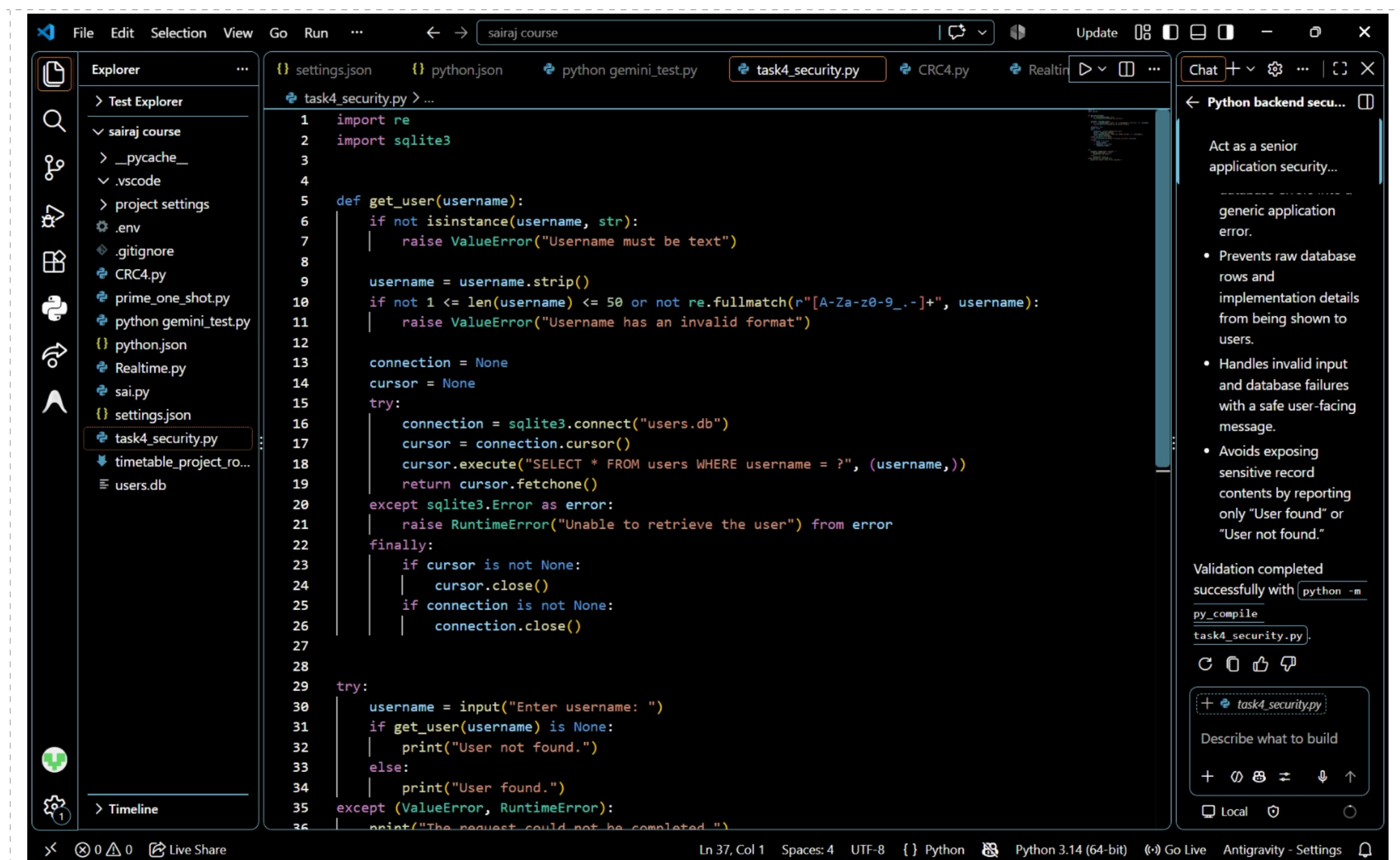
Check prime number in Python

+68 -13 · 3 wks ago

Blood pressure check code

4 wks ago

Ln 37, Col 1Spaces: 4UTF-8[] PythonPython 3.14 (64-bit)Go LiveAntigravity - Settings



7. TASK5

7.1 Objective

Use AI as an automated code reviewer to produce a structured, severity-rated quality report covering readability, naming, PEP 8 compliance, error handling, documentation, maintainability, code smells, and risk areas—not just to fix the code.

7.2 Original Code

```
def p(x):  
    a=0  
    for i in x:  
        a=a+i  
    return a  
  
def q(x):  
    print("Result is "+str(p(x)))  
  
x=[1,2,3,4,5]  
q(x)
```

7.3 AI Review Prompt

Act as an automated senior-level Python code reviewer.

Do NOT focus only on correcting the code.

Perform a structured code-quality review of the following Python program.

Evaluate the following categories:

1. Code readability
2. Naming conventions
3. Formatting and PEP 8 compliance
4. Error handling
5. Documentation quality
6. Maintainability
7. Code smells
8. Potential risks
9. Recommended improvements

For each issue provide:

- Severity: Critical / High / Medium / Low
- Category
- Problem
- Why it matters
- Recommended improvement

Then provide:

A. Overall code-quality assessment

B. Prioritized improvement plan

C. Optional improved version of the code

Clearly distinguish between actual defects and suggestions for improvement.

7.4 Structured Code Review

The AI evaluated the program across nine categories — readability, naming conventions, formatting/PEP 8 compliance, error handling, documentation quality, maintainability, code smells, potential risks, and recommended improvements — and rated each identified issue by severity (Critical / High / Medium / Low) rather than simply rewriting the code.

7.5 Code Quality Issues

Category	Issue	Severity	Recommendation
Naming	p() and q() are unclear	Medium	Use descriptive names
Naming	x, a, i lack context	Medium	Use meaningful identifiers
Formatting	Poor spacing	Low	Apply PEP 8
Documentation	No docstrings	Medium	Document functions
Maintainability	Logic is difficult to understand	Medium	Improve structure
Error handling	No validation	Low/Medium	Validate inputs

Overall Assessment: The original code is functional but has several maintainability and readability problems. The primary issues are unclear naming, lack of documentation, inconsistent formatting, and limited error-handling considerations.

7.6 Risk Areas

Lack of input validation means the function will raise an unhandled error if given a non-iterable or a list containing non-numeric values. Cryptic naming (p, q, x, a, i) increases the risk of misuse or misunderstanding by anyone maintaining the code later, since the names give no indication of intent.

7.7 Code Smells

- Cryptic function and variable names
- Missing documentation (no docstrings)
- Poor/inconsistent formatting

- Lack of separation of responsibilities (summation and display logic are tightly coupled)

7.8 Improvement Suggestions

- Rename functions and variables to describe their purpose (e.g. calculate_sum, display_result, values).
- Add docstrings to document what each function does.
- Apply PEP 8 formatting (spacing, indentation, blank lines).
- Add basic input validation where appropriate.
- Separate the summation logic from the output/printing logic.

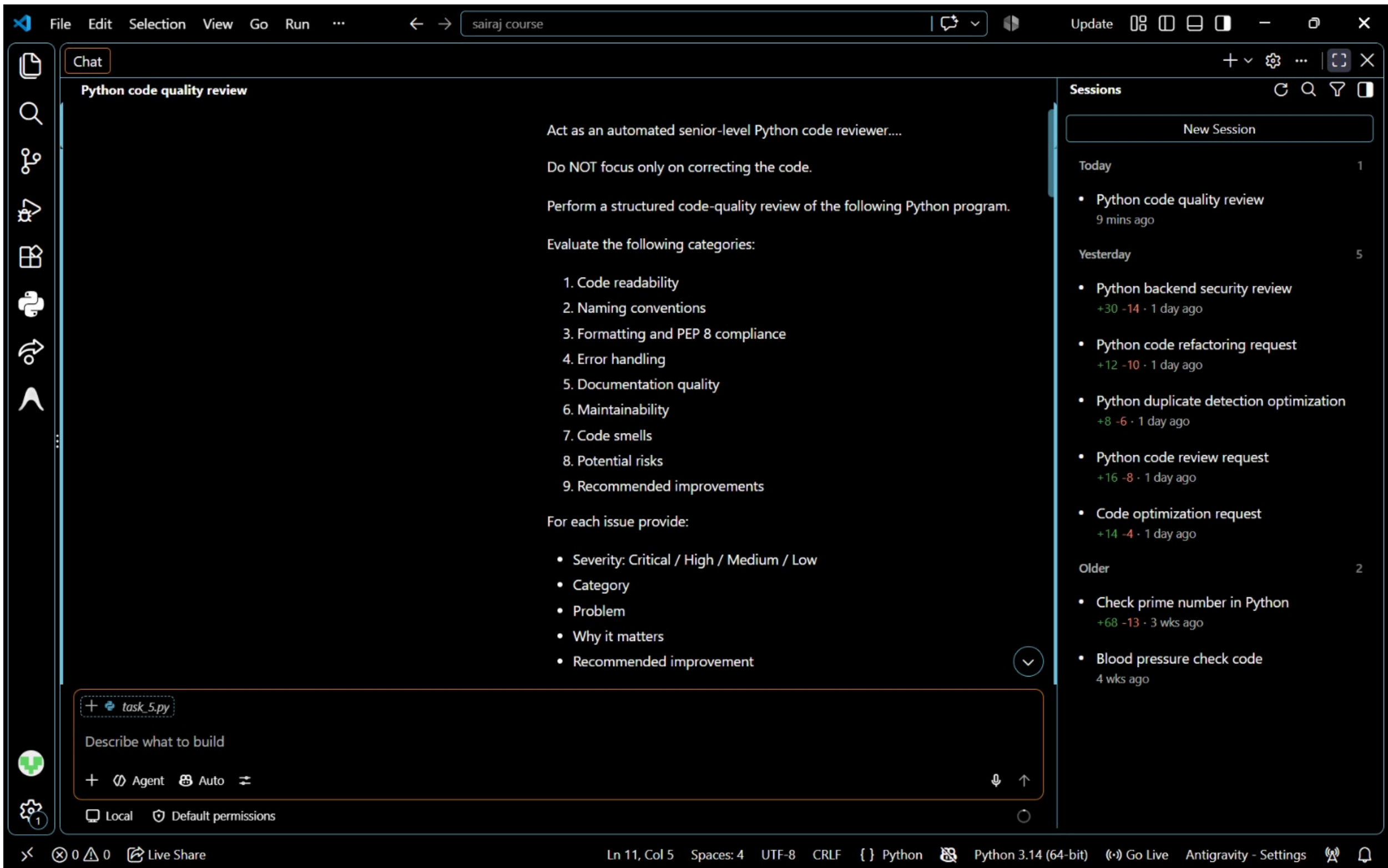
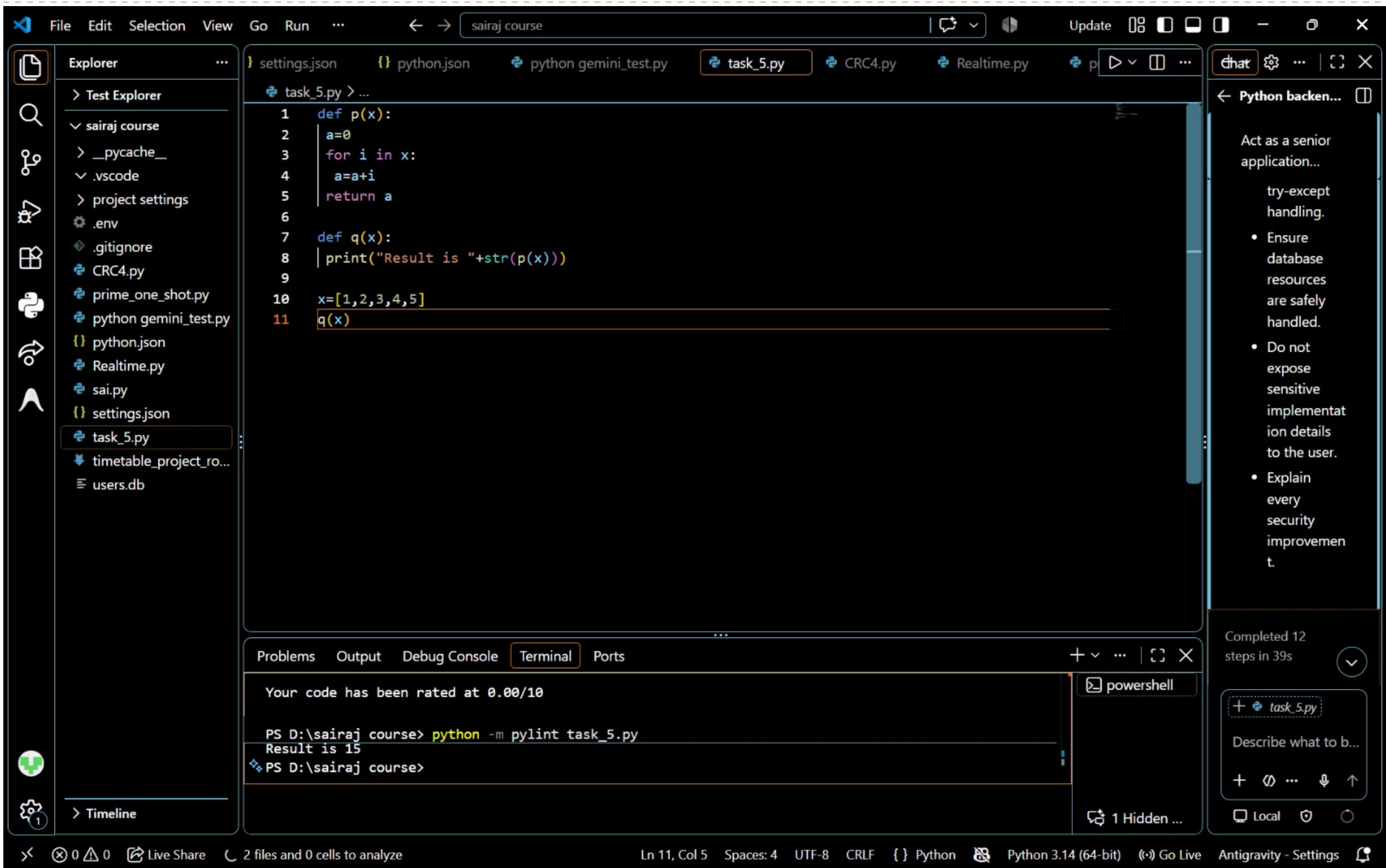
7.9 Improved Code

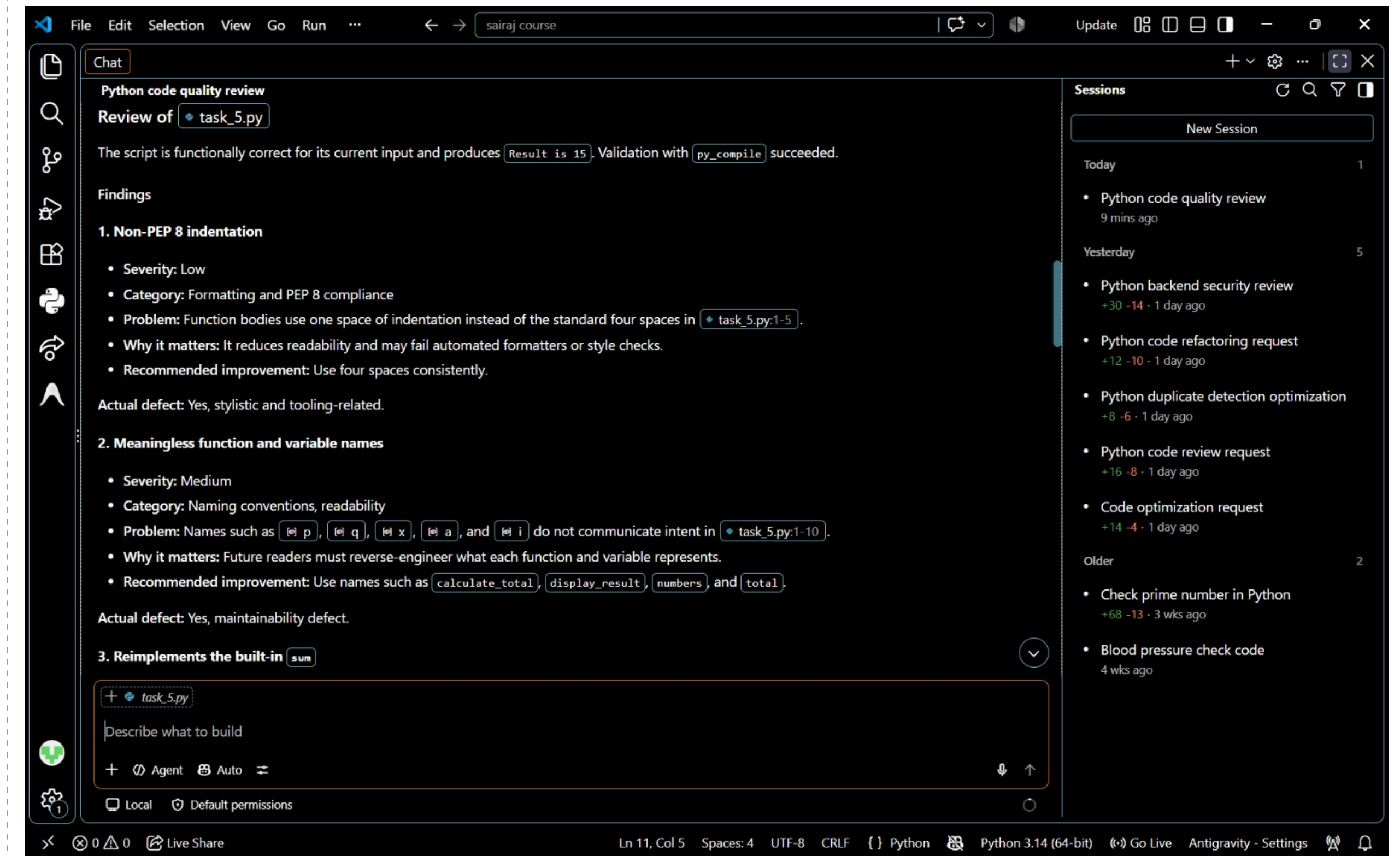
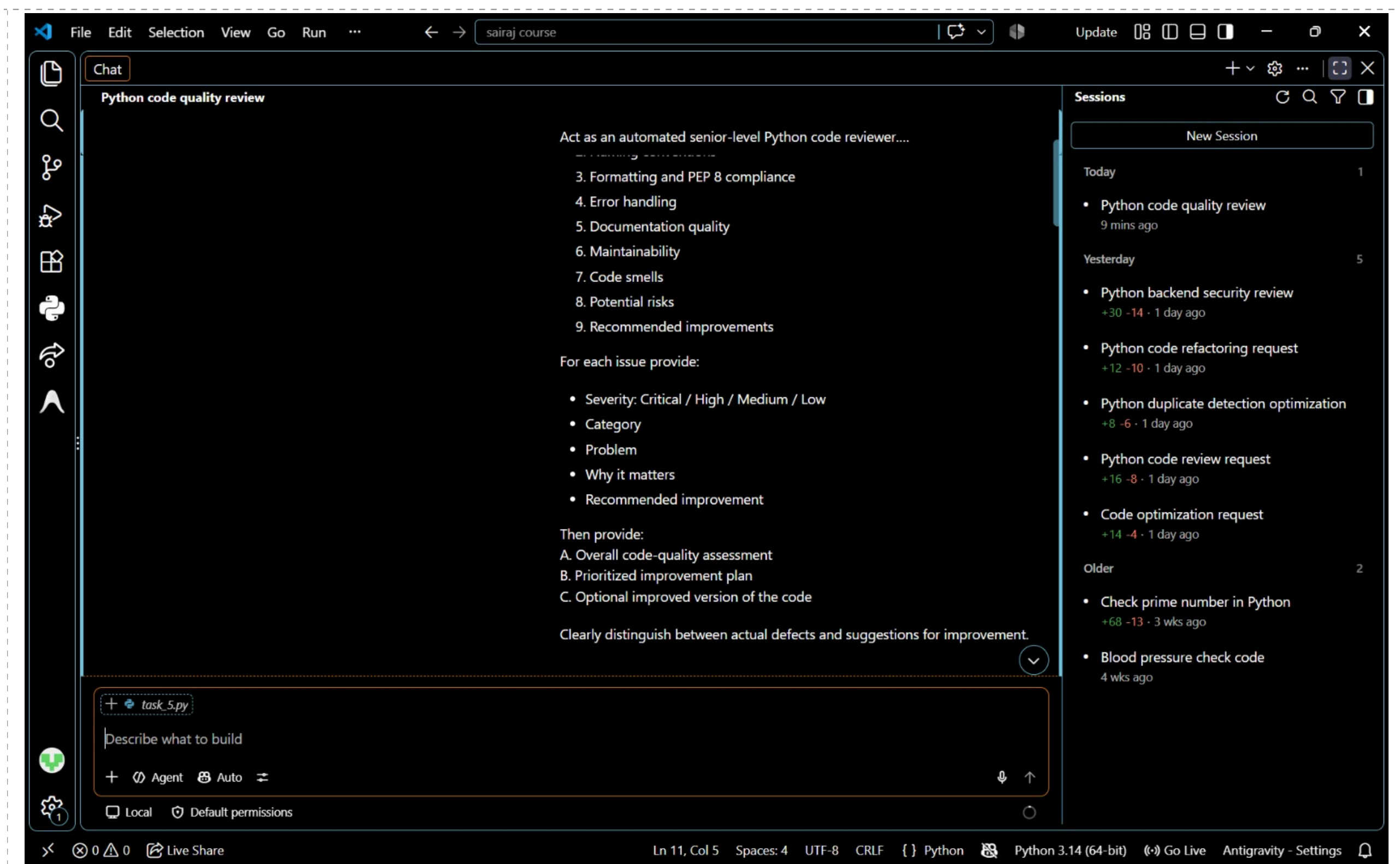
```
def calculate_sum(values):  
    """Return the sum of all values in the given iterable."""  
    total = 0  
    for value in values:  
        total += value  
    return total  
  
def display_result(values):  
    """Print the sum of the given values."""  
    print("Result is " + str(calculate_sum(values)))  
  
numbers = [1, 2, 3, 4, 5]  
display_result(numbers)
```

Expected output:

```
Result is 15
```

7.10 Screenshots





FileEditSelectionViewGoRun...<→sairaj course|🗨️🌐Update🔍📄📁🖨️-⌵🗑️X

Chat

Python code quality review

3. Reimplements the built-in `sum`

- Severity: Low
- Category: Code smells, maintainability
- Problem: `p` manually accumulates values even though Python provides `sum`.
- Why it matters: This adds code without adding behavior and creates unnecessary maintenance surface.
- Recommended improvement: Use `sum(numbers)` unless the exercise specifically requires manual iteration.

Actual defect: No functional defect; this is a code-quality suggestion.

4. No input validation or explicit error handling

- Severity: Medium
- Category: Error handling, potential risks
- Problem: The function assumes that `x` is an iterable of compatible numeric values.
- Why it matters: Invalid input can cause `TypeError`, and mixed or unexpected values may fail without a clear user-facing explanation.
- Recommended improvement: Add type hints and validate inputs where this code will receive external data. Handle expected exceptions at the application boundary.

Actual defect: Risk-dependent. It is acceptable for a controlled classroom example, but insufficient for reusable or production code.

5. Calculation and output are tightly coupled

- Severity: Medium
- Category: Maintainability, code smells
- Problem: `q` both calculates a result indirectly and prints it in `task_5.py:7-8`.

+🔗task_5.py

Describe what to build

+🔗Agent🔗Auto🔗

Local🔗Default permissions

Sessions

New Session

Today1

- Python code quality review
10 mins ago

Yesterday5

- Python backend security review
+30 -14 · 1 day ago
- Python code refactoring request
+12 -10 · 1 day ago
- Python duplicate detection optimization
+8 -6 · 1 day ago
- Python code review request
+16 -8 · 1 day ago
- Code optimization request
+14 -4 · 1 day ago

Older2

- Check prime number in Python
+68 -13 · 3 wks ago
- Blood pressure check code
4 wks ago

<⊗0⚠️0🔗Live ShareLn 11, Col 5Spaces: 4UTF-8CRLF{ }Python🔗Python 3.14 (64-bit)🔗Go LiveAntigravity - Settings🔊🔔

FileEditSelectionViewGoRun...<→sairaj course|🗨️🌐Update🔍📄📁🖨️-⌵🗑️X

Chat

Python code quality review

5. Calculation and output are tightly coupled

- Severity: Medium
- Category: Maintainability, code smells
- Problem: `q` both calculates a result indirectly and prints it in `task_5.py:7-8`.
- Why it matters: The result cannot be easily reused by another function, test, API, or user interface.
- Recommended improvement: Return the calculated value from one function and print it separately.

Actual defect: Design limitation rather than a runtime defect.

6. Hard-coded input and side effects on import

- Severity: Medium
- Category: Maintainability, potential risks
- Problem: The list is hard-coded and the script executes immediately when imported in `task_5.py:10-11`.
- Why it matters: Importing the module unexpectedly prints output and prevents reuse with other input sources.
- Recommended improvement: Put executable behavior inside a `main()` function protected by `if __name__ == "__main__":`.

Actual defect: Yes, if the file is intended to be reusable as a module.

7. Missing documentation

- Severity: Low
- Category: Documentation quality
- Problem: Functions have no docstrings, and the program has no explanation of its expected input or behavior.
- Why it matters: The code is currently simple, but documentation becomes important as behavior grows or the code is reused.

+🔗task_5.py

Describe what to build

+🔗Agent🔗Auto🔗

Local🔗Default permissions

Sessions

New Session

Today1

- Python code quality review
10 mins ago

Yesterday5

- Python backend security review
+30 -14 · 1 day ago
- Python code refactoring request
+12 -10 · 1 day ago
- Python duplicate detection optimization
+8 -6 · 1 day ago
- Python code review request
+16 -8 · 1 day ago
- Code optimization request
+14 -4 · 1 day ago

Older2

- Check prime number in Python
+68 -13 · 3 wks ago
- Blood pressure check code
4 wks ago

<⊗0⚠️0🔗Live ShareLn 11, Col 5Spaces: 4UTF-8CRLF{ }Python🔗Python 3.14 (64-bit)🔗Go LiveAntigravity - Settings🔊🔔

FileEditSelectionViewGoRun

sairaj course

Update

Chat

Python code quality review

- Recommended improvement: Add concise docstrings and type annotations.

Actual defect: Documentation gap, not a runtime defect.

8. No tests

- Severity: Medium
- Category: Maintainability, potential risks
- Problem: There are no automated tests for empty input, negative values, invalid values, or normal totals.
- Why it matters: Future changes could break behavior without detection.
- Recommended improvement: Add focused tests for normal, empty, and invalid inputs.

Actual defect: Process gap; low risk for this tiny script, higher risk as functionality expands.

A. Overall Assessment

Overall quality: Basic and functional, but not production-ready.

The program is easy to execute and produces the expected result. Its main weaknesses are opaque naming, nonstandard formatting, lack of separation between computation and presentation, missing documentation, and no protection against side effects during import.

There are no critical defects in the current controlled use case.

B. Prioritized Improvement Plan

- Fix indentation and apply standard formatting.
- Rename functions and variables to express intent.
- Separate calculation from printing.

+ task 5.py

Describe what to build

+ Agent Auto

Local Default permissions

Sessions

New Session

Today1

- Python code quality review10 mins ago

Yesterday5

- Python backend security review+30 -14 · 1 day ago
- Python code refactoring request+12 -10 · 1 day ago
- Python duplicate detection optimization+8 -6 · 1 day ago
- Python code review request+16 -8 · 1 day ago
- Code optimization request+14 -4 · 1 day ago

Older2

- Check prime number in Python+68 -13 · 3 wks ago
- Blood pressure check code4 wks ago

Ln 11, Col 5Spaces: 4UTF-8CRLF{ } PythonPython 3.14 (64-bit)Go LiveAntigravity - Settings

FileEditSelectionViewGoRun

sairaj course

Update

Chat

Python code quality review

- Category: Maintainability, potential risks
- Problem: There are no automated tests for empty input, negative values, invalid values, or normal totals.
- Why it matters: Future changes could break behavior without detection.
- Recommended improvement: Add focused tests for normal, empty, and invalid inputs.

Actual defect: Process gap; low risk for this tiny script, higher risk as functionality expands.

A. Overall Assessment

Overall quality: Basic and functional, but not production-ready.

The program is easy to execute and produces the expected result. Its main weaknesses are opaque naming, nonstandard formatting, lack of separation between computation and presentation, missing documentation, and no protection against side effects during import.

There are no critical defects in the current controlled use case.

B. Prioritized Improvement Plan

- Fix indentation and apply standard formatting.
- Rename functions and variables to express intent.
- Separate calculation from printing.
- Add type hints and a `main()` entry point.
- Replace the manual loop with `sum` unless manual iteration is required.
- Add tests if the code will be reused or extended.
- Add input validation if values can come from users, files, or external systems.

+ task 5.py

Describe what to build

+ Agent Auto

Local Default permissions

Sessions

New Session

Today1

- Python code quality review11 mins ago

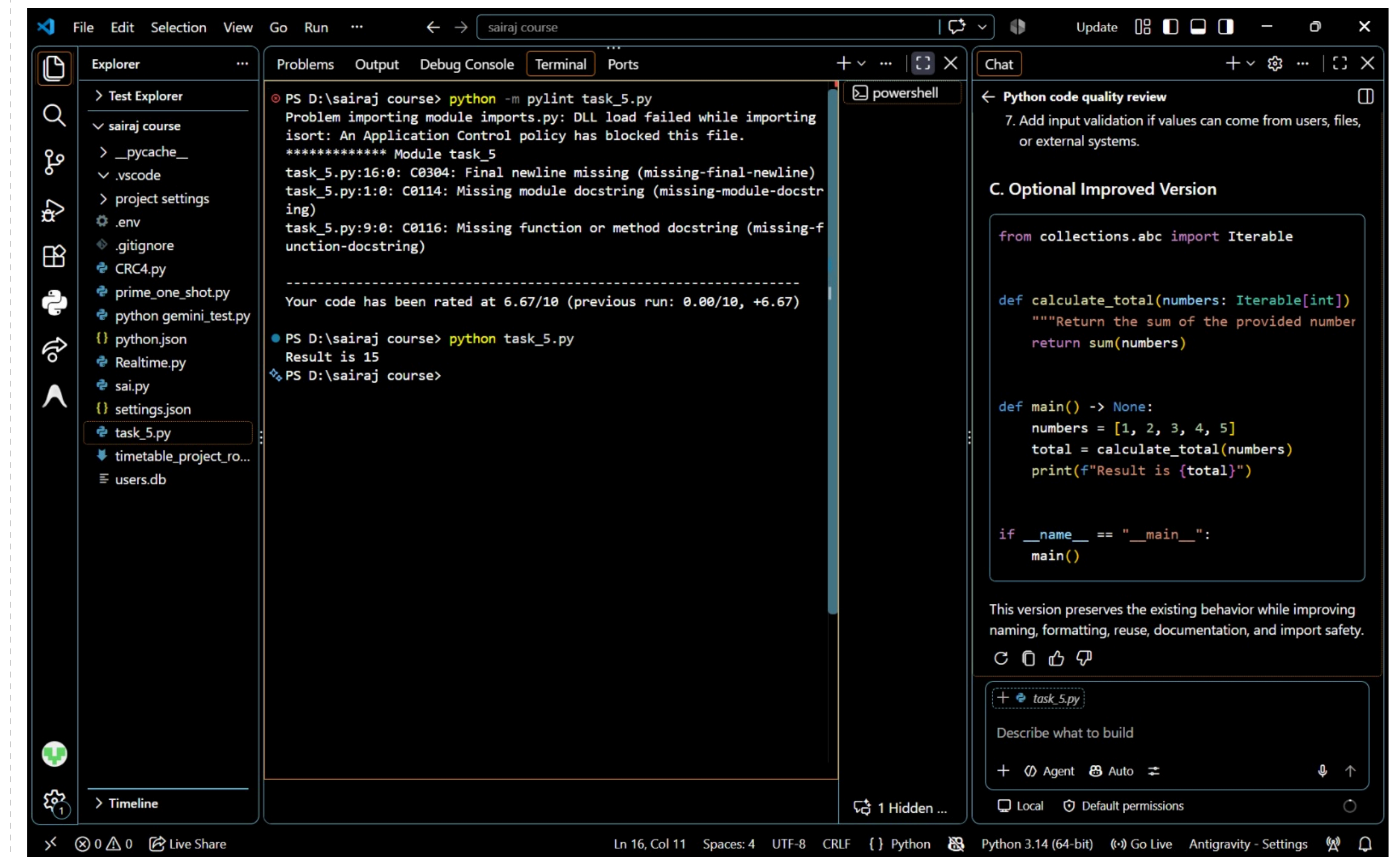
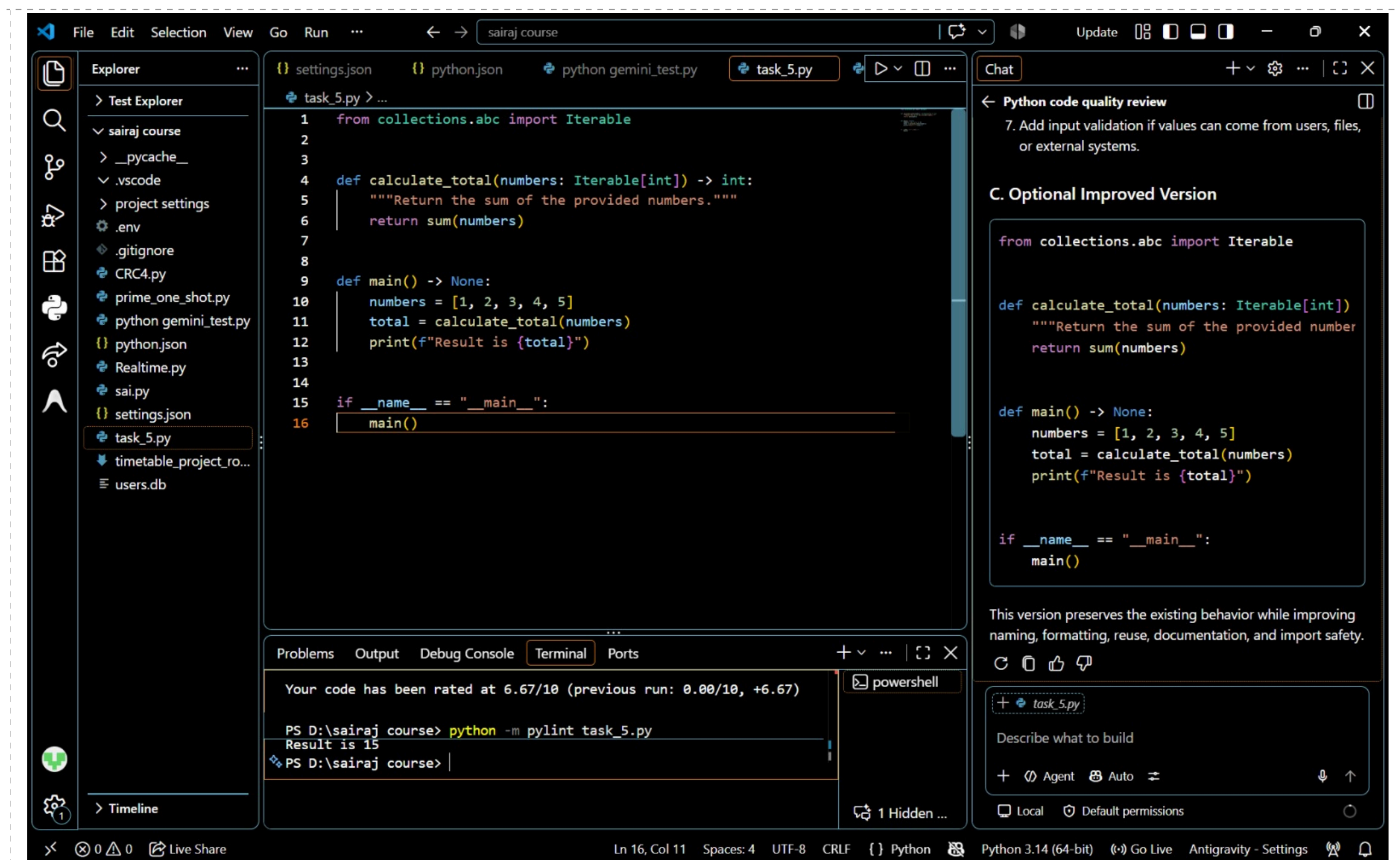
Yesterday5

- Python backend security review+30 -14 · 1 day ago
- Python code refactoring request+12 -10 · 1 day ago
- Python duplicate detection optimization+8 -6 · 1 day ago
- Python code review request+16 -8 · 1 day ago
- Code optimization request+14 -4 · 1 day ago

Older2

- Check prime number in Python+68 -13 · 3 wks ago
- Blood pressure check code4 wks ago

Ln 11, Col 5Spaces: 4UTF-8CRLF{ } PythonPython 3.14 (64-bit)Go LiveAntigravity - Settings



8. OVERALL CONCLUSION

Across all five tasks, AI assistance was effective at catching syntax and logic errors, identifying an inefficient nested-loop algorithm and suggesting a hash-based (set) optimization, improving naming, documentation, and PEP 8 compliance, detecting security vulnerabilities such as SQL injection and recommending parameterized queries with input validation and exception handling, and producing a structured, severity-rated automated code review report.

Overall, using AI as a code reviewer — rather than only as a code generator — helped produce Python code that is more correct, efficient, readable, secure, and maintainable, and reinforced the importance of PEP 8 and software-engineering best practices throughout the development process.